

Sprawozdanie

z projektu Symulatora Układu Automatycznej Regulacji – Sieci Komputerowe

Skład sekcji:

Martin Walldorf – realizacja funkcjonalności sieciowych całego projektu

Opis działania protokołu komunikacyjnego

Komunikacja sieciowa opiera się na architekturze klient-serwer zaimplementowanej w klasie NetworkManager przy użyciu klas QTcpServer i QTcpSocket.

TCP jest protokołem strumieniowym i nie gwarantuje zachowania granic wiadomości, dlatego zaimplementowano jawne flagowanie długości pakietu.

Każda paczka danych wysyłana przez sieć ma następującą strukturę binarną:

1. **Rozmiar danych (quint32):** Pierwsze 4 bajty określają, jak wielka jest właściwa paczka danych.
2. **Tablica bajtów (QByteArray):** Właściwy ładunek paczki.

Wewnątrz właściwego ładunku paczki (QByteArray) pierwszym bajtem jest identyfikator typu wiadomości (quint8), który definiuje jak należy interpretować resztę danych ładunku:

Typ (quint8)	Kierunek	Zawartość	Opis
1	W obie strony	JSON	Stan konfiguracji (nastawy PID, parametry ARX i generatora)
2	Regulator -> Obiekt	double u, double wz	Sygnał sterujący i wartość zadana
3	Obiekt -> Regulator	double y	Wartość regulowana
4	Regulator -> Obiekt	bool running	START/STOP
5	Regulator -> Obiekt	brak	Reset symulacji

Opis działania serializacji

Do przesyłania konfiguracji służy serializacja tekstowa polegająca na tym, że stan trzech głównych komponentów przesyłany jest w jednym obiekcie JSON, który zawiera:

Model ARX – współczynniki wielomianów A i B, opóźnienie transportowe oraz zakłócenie.

Regulator PID – nastawy Kp, Ti, Td i typ całki

Generator – okres, amplituda, stała, wypełnienie oraz typ kształtu sygnału

Deserializacja

Działa w sposób odwrotny względem serializacji. Odczytuje obiekt JSON i konfiguruje wszystkie parametry układu automatycznej regulacji.

Wykrywanie gubienia pakietów

Wykrywanie to opiera się na analizie czasu rzeczywistego przesyłania paczek (odstępów pomiędzy kolejnymi paczkami sieciowymi) i porównywaniu go z interwałem timera symulacji sygnału.

Pomiar czasu: Przy każdym odebraniu pakietu typu 3 (tabelka), aplikacja sprawdza aktualny znacznik czasu (timestamp) w milisekundach i oblicza różnicę (deltę) od poprzedniego pakietu.

Kryterium błędu: Jeśli rzeczywisty czas, który upłynął od poprzedniej paczki, jest większy niż 120% oczekiwanego interwału aplikacja uznaje, że doszło do zgubienia pakietu w warstwie aplikacji.

Sygnalizacja: Interfejs użytkownika natychmiast wyświetla ostrzeżenie:

"Wykryto gubienie pakietów!".

Jeśli transmisja wraca do normy, komunikat znika i pojawia się komunikat o prawidłowej pracy regulatora – **Tryb Sieciowy**.

Historia rozwoju aplikacji o funkcjonalności sieciowe

- Napisanie klasy NetworkManager, która pozwoliła na uniezależnienie od siebie procesu regulacji i symulacji obiektu
- Zaimplementowanie mechanizmu ramkowania transportowego i systemu identyfikatorów (quint8 o wartościach od 1 do 5) (tabelka)
- Wprowadzenie pełnej serializacji parametrów UAR przy użyciu formatu JSON. Po zmianie nastaw cała struktura parametrów jest natychmiast wysyłana z jednej instancji do drugiej dzięki czemu obie instancje są zawsze zsynchronizowane
- Dodanie obsługi zdarzeń utraty połączenia, dzięki czemu po przejściu symulacji z trybu sieciowego w tryb stacjonarny jest ona kontynuowana na lokalnych ustawieniach na obydwu instancjach
- Dodanie wykrywania i sygnalizacji gubienia pakietów oraz powrotu do prawidłowej pracy.

Najistotniejsze napotkane trudności w realizacji tematu

Zjawisko uderzenia sygnałów przy utracie połączenia - Nagłe rozłączenie sieci i powrót do trybu stacjonarnego powodowały drastyczne skoki wartości sterujących. Rozwiązaniem było zaimplementowanie mechanizmu bezuderzeniowego przełączania realizowane poprzez śledzenie stanu. W momencie wykrycia utraty połączenia aplikacja przechwytyje ostatnie poprawne stany sieciowe: sygnał sterujący **u**, wyjście obiektu **y**, uchyb **e** oraz aktualną wartość całki. Następnie, aż do ponownego uruchomienia lokalnego timera, stany te są sztucznie wymuszane.

Problem asymetrii ról i wielodostępności interfejsu - W trybie sieciowym jedna instancja programu staje się Regulatorem, a druga Obiektem. Gdyby użytkownik po stronie Obiektu próbował zmienić np. nastawę **Kp**, doprowadziłoby to do rozbieżności matematycznej między dwoma połączonymi programami. Rozwiązaniem okazało się wprowadzenie synchronizacji sieciowej JSON. W zależności od przyjętej roli sieciowej, funkcja **updateUIMode()** blokuje lub odblokowuje odpowiednie grupy kontrolek (np. w trybie Obiektu pole nastaw PID i przycisk Start/Stop stają się nieaktywne). Dodatkowo, każda zatwierdzona zmiana konfiguracji po dowolnej stronie (np. zmiana struktury wielomianu w oknie ustawień ARX powoduje automatyczne wygenerowanie paczki tekstowej JSON, przesłanie jej jako pakiet typu 1 (tabelka) i natychmiastowe nadpisanie parametrów u partnera sieciowego.

Wykrywanie gubienia pakietów - Opóźnienia lub chwilowe gubienie pakietów destabilizowały działanie pętli sprzężenia zwrotnego, fałszując wyniki algorytmów różniczkujących i całkujących. Rozwiązaniem było wprowadzenie asynchronicznej kaskady zdarzeń oraz licznika czasu rzeczywistego. Zamiast sztywnego wywoływania obliczeń przez zegar próbkowania, Regulator wysyła sygnał sterujący **u** i „czeka” na asynchroniczną odpowiedź z obiektu. Dodatkowo przy każdym odebraniu pakietu zwrotnego obliczana jest różnica czasu. Jeśli symulacja działa z wysoką częstotliwością, a rzeczywisty czas dostarczenia pakietu przekroczy 120% założonego interwału, system natychmiast zgłasza to w GUI z ostrzeżeniem o gubieniu pakietów

"Wykryto gubienie pakietów!".

Czego się nauczyłem?

Opanowałem implementację protokołów strumieniowych, sygnałów i slotów obsługujących zdarzenia sieciowe oraz synchronizację procesów, które nie współdzielią pamięci i zależą od asynchronicznych odpowiedzi z sieci.

Zrozumiałem jak ważne jest uwzględnianie w programie stanów krytycznych w sieciach komputerowych i dlatego nauczyłem się pisanie kodu w taki sposób, aby te sytuacje wyeliminować lub błyskawicznie na nie zareagować.

Nauczyłem się jak rozdzielić odpowiedzialność pomiędzy instancje regulatora a instancje obiektu w układach sieciowych oraz wprowadzić funkcjonalności takie jak blokowanie kontrolek czy zapobieganie powstawania tzw. „wyścigów stanów”.